Behique: making a check fast

Written 2026-10-31. Before and after numbers are both measured on the same six repos (section 2 before, section 6 after): a check is 3.2x faster overall, test runs 9x to 20x.

1. The question asked

"Step 5, run the check, takes 1 to 38 minutes. Can we use Firecracker?"

Short answer: Firecracker would not make a check faster. Firecracker is a way to start a small virtual machine in about 125 ms so that untrusted code is walled off by hardware. It replaces the *container* around the work, not the work. Inside that VM, `npm ci` and `next build` take exactly as long as they do in Docker, and VM disk I/O is usually a little slower. Firecracker is the answer to "plain Docker is not a hard security wall" (a known limit in docs/sandbox.md), and it is a real, separate project: root-filesystem images, tap networking, a jailer, no Docker image ecosystem. It is kept here as the last, optional phase, for isolation, and measured for its cost when we get there.

What makes a check slow is below, and it is fixable without changing the sandbox technology.

2. Where the time goes (before)

Measured from the event timestamps of the final run of each repo in the six-repo bench (2026-10-30, sandbox on hardac2, each result is one run).

Repo	Crawl	Compare phase (builds)	Test runs (builds)	Per test run	Screenshots	Total
nextjs-blog	99 s	464 s (4)	1,550 s (30)	50 s	146 s	36 min
umami	7 s	266 s (4)	1,327 s (18)	75 s	204 s	30 min
excalidraw	120 s	229 s (4)	1,082 s (30)	39 s	108 s	24 min
vitepress	85 s	391 s (4)	189 s (6)	26 s	84 s	11 min
todomvc (static)	100 s	321 s (9)	208 s (30)	7 s	20 s	9 min
html5-boilerplate (static)	1 s	76 s (4)	none			1.3 min

How to read it:

- **Every test run is a brand-new build of the same commit.** The rule "each run of each test gets a freshly seeded build" is right (one test must not change what another sees), but in the sandbox a fresh build means tar the checkout, create a volume, seed it, run `npm ci`, run the app's build step, start the app, open a tunnel, wait for it to answer. nextjs-blog does that 37 times for two commits. Nothing is kept between builds.
- **The per-build floor is 7 s** (todomvc: nothing to install, so this is tar + volume + seed + container + tunnel + ready probe). Everything above that is install and build.
- **Nothing runs in parallel.** Test runs, screenshots and bisect steps are strictly one after another. hardac2 has 24 cores and 27 GB free.
- **The crawl is 85 to 120 s** and runs on every check of the same good version.
- The compare phase (4 builds plus recordings) is 4 to 8 minutes; todomvc needed 9 builds because the control had to be recorded five extra times to learn its noise.
- Bisect (not in the bench numbers) is about 14 more builds when it runs: 8 steps and 6 verify runs.

The dashboard today has no idea which of these it is waiting on: the record has no timing events, so the table above had to be computed from event timestamps by hand.

3. The plan: phases

Each phase is a Muse spec in `docs/specs/speed/`, buildable by its own agent, offline-tested with fakes, then measured on the real repos. Phases 1, 2 and 4 touch different files and can be built in parallel.

Phase 0, by hand first: measure inside the tool

Record `build.ready` events (`ref`, `sha`, `ms`, `cached: true|false`, `phase: compare|test|shots|bisect`) and a `check.timing` summary at `case.closed`, so the before/after is measured by Behique and shown in the dashboard and the bench table, not computed by hand. `bench/check-bench.mjs` gains per-phase seconds columns. Nothing else changes. This is what makes the after numbers trustworthy.

Phase 1: a build cache per commit (the big one)

After the first successful install and build of a commit, keep the result as a Docker image: `docker commit` the work container to `behique/<app>:<sha12>-<fingerprint8>` where the fingerprint covers the profile's install and build commands, the base image and the env. The next `startBuild` for the same key skips tar, volume, seed, install and build: it creates a fresh container from that image and starts the app. Overlay2 gives every container its own copy-on-write filesystem, so each run still starts from exactly the state a fresh install produces, and nothing a test writes survives into the next run. Sidecars (Postgres and so on) stay fresh per run as now.

- Owns: `src/release/sandbox/cache.mjs` (new), additions to `docker-args.mjs` (`commitArgs`, `imageListArgs`, `imageInspectArgs`, `imageRemoveArgs`; image names only from validated parts, labelled `behique`), `sandbox.mjs` (`prepareFromImage`), `backend.mjs` (look up, then build and commit), `clean.mjs` (prune: keep the newest N images per app, drop images older than D days, never touch unlabelled images).
- Rule: a cache hit must be reported in the evidence (`cached: true`), and a person can turn the cache off per run (`--no-cache`, and a box in New run) for a fully fresh check.
- Expected: per test run falls from 26 to 75 s to about the 5 to 7 s floor (the first build of each commit still pays full price). nextjs-blog from about 36 min to about 8 min. todomvc barely changes (it is already at the floor).

Phase 2: run tests and screenshots in parallel

`confirm.mjs` runs the test runs through a small pool (default 3, from Settings) on distinct ports (8300 + k) and distinct sandbox run ids; evidence events are appended as runs complete and judging happens after all of them. Screenshots (3 builds) and the bisect verify step (6 runs) use the same pool; bisect steps stay serial because each depends on the last. Pool size is capped by memory: floor(free memory / per-app memory limit), so three 10 GB Next.js builds do not fight for 27 GB.

- Owns: `src/release/confirm.mjs` (the run loops), `src/release/pool.mjs` (new), `bisect.mjs` (verify only), Settings (pool size).
- Expected: the test-run and screenshot phases divide by the pool size.

Phase 3: cut the per-build floor

With Phase 1 the floor (7 s) becomes the biggest per-run cost. Likely cuts, to be measured one by one: reuse one ssh connection for all docker commands and tunnels (ControlMaster) instead of a new one per build; create the app container directly from the cached image with no volume; shorten the ready probe interval. Target: 2 to 3 s per run. This phase is "measure the floor, cut what is left", and it may be small.

Phase 4: do not crawl the same good version twice

Cache the crawl result per (app, good sha, crawl options hash) under `out/crawl/`. Checking a new candidate against the same good version skips the 85 to 120 s crawl; a person can force a new crawl. Owns: `src/release/check-app.mjs`, `crawl/cli.mjs`.

Phase 5, optional and separate: stronger isolation

If a harder wall than plain Docker is wanted, the cheap path is a Docker *runtime* that keeps the whole sandbox code as it is: gVisor (`runsc`) or Kata Containers (which runs each container in a Firecracker or QEMU micro-VM; hardac2 has KVM). Both are `--runtime` flags on the same `docker create`. Both cost speed (gVisor slows file I/O; Kata adds VM boot and memory). This phase measures that cost on the six repos and the decision is made with numbers. Raw Firecracker without Kata is not recommended: weeks of plumbing for the same wall.

4. What "after" should look like

Estimates, to be replaced by measurements. A check = crawl + compare + tests + screenshots, no bisect.

Repo	Before	After Phase 1 (cache)	After Phases 1+2 (cache, pool of 3)
nextjs-blog	36 min	about 8 min	about 5 min
umami	30 min	about 7 min	about 4 min
excalidraw	24 min	about 7 min	about 4 min
vitepress	11 min	about 6 min	about 4 min
todomvc	9 min	about 8 min	about 6 min

The compare phase dominates what is left: four recordings of the app (each a real browser walking 40 to 50 steps) plus the first install of each commit. That is the cost of the method, not waste.

5. Other slowness found while looking

- **Extra control recordings** (todomvc: 5, each a build plus a 40-step recording) are the price of learning noise. Caching the good build (Phase 1) makes each one cheap; the recording itself stays.
- **The dashboard polls** `/api/ui/runs/current` and re-reads the whole cases list; `/api/ui/cases` builds a view-model for all 71 reports on every call (0.17 s now, grows with reports). Cheap to cache per file mtime; worth doing when it passes a second.
- **The ready probe** retries every 300 ms up to the profile's timeout; fine.
- **Install stalls**: the watchdog (8 min idle, one retry) is in; a stall still costs up to 16 min once. A registry mirror or an npm cache volume on hardac2 would cut both the stall risk and first-install time; worth adding to Phase 1 as a labelled cache volume mounted read-write at `/root/.npm` (and the pnpm and yarn equivalents).
- **Model calls** are not on the path: a check makes none.

6. After: measured 2026-10-31 on hardac2

Same command, same six repos, same versions, after Phases 0, 1, 2 and 4 were built (Phase 3 and Phase 5 were not). Pool of 3, memory limit 10 GB per build. The cache was empty except for html5-boilerplate (a smoke run minutes earlier), so every first build of a commit still installed and committed; a second run of the same pairs would start every build from the cache.

Repo	Before	After	Faster by	Compare s (builds, cached)	Tests s (builds, cached)	Shots s (builds, cached)	Outcome
nextjs-blog	36 min	9.6 min	3.7x	396 (8, 6)	167 (30, 30)	15 (3, 3)	confirmed 5/5 (same)
umami	30 min	6.8 min	4.4x	332 (5, 3)	67 (18, 18)	9 (3, 3)	confirmed 3/3 (same)
excalidraw	24 min	7.2 min	3.3x	294 (5, 3)	123 (30, 30)	16 (3, 3)	confirmed 5/5 (same)

Repo	Before	After	Faster by	Compare s (builds, cached)	Tests s (builds, cached)	Shots s (builds, cached)	Outcome
vitepress	11 min	4.7 min	2.3x	238 (6, 4)	32 (12, 12)	10 (3, 3)	confirmed 1/2
todomvc (static)	9 min	6.2 min	1.5x	296 (7, 5)	64 (30, 30)	9 (3, 3)	confirmed 4/5 in this run; 5/5 after the fix below
html5-boilerplate (static)	1.3 min	0.24 min	5.4x	13 (4, 4)	none	none	no_change (same)
All six	**111 min**	**35 min**	**3.2x**				5 confirmed, 1 no change (same)

What the columns say:

- **Tests** went from the biggest cost to a small one: nextjs-blog's 30 test runs took 1,550 s before (one build each, about 50 s) and 167 s after (every build from the cache, three at once): 9x. umami 1,327 s to 67 s (20x). Every test-run build in the six checks was a cache hit (123 of 123).
- **Compare** is now most of a check (60 to 90 percent). It still has the first install of each commit (the two uncached builds in each row; nextjs-blog's `next build` is most of its 396 s), and its four to eight recordings run one after the other. Phase 3 and running the compare recordings through the same pool are the next cuts; the estimate table in section 4 assumed compare stayed as it was, and that is what happened.
- **Screenshots** went from 84 to 204 s to 9 to 16 s: three cached builds at once.
- **Crawl**: nextjs-blog, excalidraw and html5-boilerplate reused the crawl saved by the earlier bench (0 s instead of 99, 120 and 1 s); vitepress, umami and todomvc had no saved crawl of that commit and crawled as before. Their totals above include that crawl.
- **Outcomes** are the same at the app level (five confirmed, one no change). Two findings read differently from earlier runs, todomvc 4 of 5 confirmed (5 of 5 on 2026-10-30) and vitepress 1 of 2, so both were rerun the same day with `BEHIQUE_POOL=1`: todomvc came back 5 of 5, vitepress stayed 1 of 2.
- **todomvc R4 was the pool's fault, and it was deterministic, not noise.** The fact is the tweet button's iframe `src`, which embeds the page's own address (`original_referer=http://127.0.0.1:8300/`). The compare phase records on port 8300; the pool serves slots 1 and 2 on 8301 and 8302, so on those slots the expected substring could never match whatever the app did (the good version passed only the one run that landed on slot 0). Fix: a finding now carries the origin it was recorded at, and where an expected value contains that origin, plain or URL-encoded, the generated test swaps in the origin of the build it is talking to (`const live = new URL(page.url()).origin` in `src/export.mjs`, tests in `src/export.test.mjs`). Rerun with the pool on after the fix: **todomvc 5 of 5, R4 passed 3 of 3 on the good version**, 5.8 min (tests 74 s for 30 builds, against 175 s with the pool off). Of the 13 pairs checked so far, R4 was the only exported test that embedded the port; any app that writes its own origin into the page would have hit this.
- **vitepress R2 is a third-party ad, not the pool.** The fact is the Carbon Ads image `src`, which the ad server rotates between creatives. The 10-30 run never saw it change; on 10-31 the good and candidate recordings happened to get different creatives, which made R2 a finding at all, and the good version then failed 2 of 3 runs on a third creative. In the pool-off rerun all six runs got one creative and R2 was simply not reproduced. The confirmed count is 1 both days. Treating content from third-party hosts as volatile is a separate improvement, not part of this plan.
- With a warm cache the compare phase barely moved (vitepress 238 s to 253 s, todomvc 296 s to 306 s with every build cached), which confirms that compare is recording time, not install time.

What a second run of the same pairs would add: the first build of each commit comes from the cache too, so compare drops by one install per commit (nextjs-blog by roughly 100 to 150 s more). The cache on hardac2 after this bench: 12 images, each about 1.2 GB as listed but sharing the 1.1 GB base image's layers; `node src/release/sandbox/clean.mjs --target ssh://hardac2` keeps the newest 4 per app and drops the rest after 14 days.

Not built: Phase 3 (floor cuts) and Phase 5 (gVisor or Kata, only if a harder wall than Docker is wanted; to be measured, not assumed).